

0.75. Arrivé en (0, 0), le A devient B

Le A part de (10, 0) et suit la croix ; sa position s'affiche en direct. Quand il arrive en (0, 0), il se transforme en B : `if (x == 0 && y == 0)`.

1. Ce qu'il faut comprendre

Le A part de (10, 0), en haut de l'écran, et se déplace avec la croix, comme au 0.71. Sa position s'affiche **en direct** en bas de l'écran, comme au 0.68 : `x` à gauche, `y` à côté.

Ce qui est nouveau ici : une condition sur la position. `if (x == 0 && y == 0)` veut dire : « si `x` vaut 0 **ET** si `y` vaut 0 ». `&&` se lit « et » : il faut que **les deux** soient vrais en même temps. En (0, 5), `x == 0` est vrai mais `y == 0` est faux : rien ne se passe. En (0, 0), les deux sont vrais : la condition est vraie.

La lettre dans une variable : `lettre` vaut 0 au départ, et la croix pose `ALPHABET[lettre]`, donc `ALPHABET[0]`, le A. Quand la condition est vraie, `lettre = 1;` : à l'image suivante, `deplace_croix` pose `ALPHABET[1]`, le B.

Le B reste un B : rien ne remet `lettre` à 0. Même en quittant la case (0, 0), la lettre reste transformée.

Deux façons d'écrire le `if` : sur une ligne, `if (x == 0 && y == 0) lettre = 1;`, ou **avec des accolades** : `if (x == 0 && y == 0) { lettre = 1; }`. Les deux font exactement la même chose. Sans accolades, le `if` ne commande **qu'une seule** instruction, celle qui le suit. Avec des accolades, il commande **tout ce qui est entre elles** : on peut y mettre plusieurs lignes (changer la lettre ET jouer un son, par exemple). Le programme l'écrit avec des accolades, et montre l'autre forme en commentaire juste au-dessus.

Pour y arriver : la flèche **gauche**, dix pas. Les nombres en bas montrent `x` qui descend : 010, 009... jusqu'à 000. Et la lettre change.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1254 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      75 o ← la console, à l'allumage — la mise en route de la console
deplace_croix() 217 o ← main() — une fonction fournie par le langage
main()          70 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o ← image() — attend l'image suivante
AttendreVBlank  16 o ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
```

AdresseCase	10 o	← effacer(), poser()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	704 o	← CopierTuiles	– la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

bouton()	× 4	20 o sur place, dans deplace_croix()	+ LireManette (37 o)
nombre()	× 2	20 o sur place, dans main()	+ EcrireNombre (33 o)
effacer()	× 1	15 o sur place, dans deplace_croix()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans deplace_croix()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()	

Tes fonctions appelées :

deplace_croix()	× 1	21 o sur place, dans main()
-----------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <nombre>           // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 10;           // la COLONNE : le A part de la colonne 10...
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;           // ...et de la ligne 0, en haut de l'écran
```

4. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;      // QUELLE lettre : 0 = A, 1 = B
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

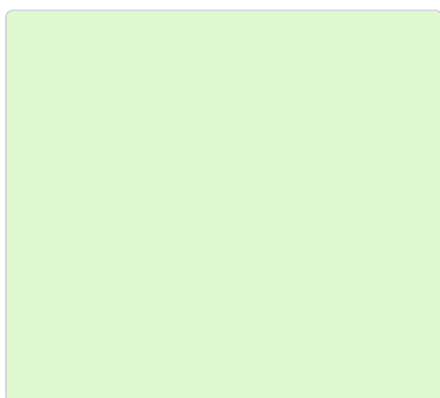
        // La croix déplace la lettre ; ALPHABET[lettre] : le A, puis le B.
        deplace_croix(x, y, ALPHABET[lettre], 250);

        // Le contrôleur de position : x et y, en direct, en bas de l'écran.
        nombre(0, 17, x);
        nombre(4, 17, y);

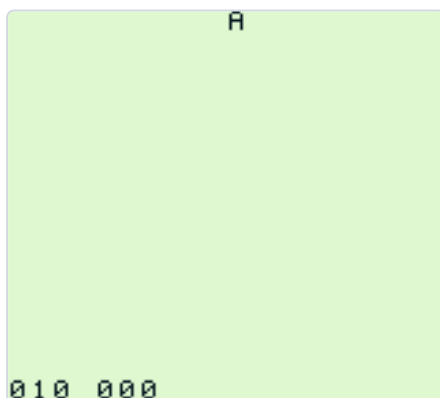
        // La condition, morceau par morceau :
        //
        //   if (x == 0 && y == 0) lettre = 1;
        //   |         | |         |
        //   |         | |         +-- alors : la lettre devient ALPHABET[1], le B
        //   |         | +----- y vaut-il 0 ? (tout en haut)
        //   |         +----- && : ET – les DEUX doivent être vrais
        //   +----- x vaut-il 0 ? (tout à gauche)
        //
        //   en (10, 0) : x == 0 faux          → rien
        //   en (0, 5)  : x == 0 vrai, y == 0 faux → rien
        //   en (0, 0)  : les deux vrais        → le A devient B
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A en haut de l'écran ; avec la flèche gauche, il va jusqu'au coin (0, 0), et là il devient un B. En bas, sa position en direct.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
 - ✓ en route, c'est encore un A ($x \neq 0$)
 - ✓ en (0, 0), il est devenu B
 - ✓ il reste un B en repartant (en (0, 3))
 - ✓ la position s'affiche
-

Ensuite : **0.75.1. Arrivé en (0, 0), le A devient B — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++